International Islamic University Chittagong
Department of Computer Science & Engineering

Course Code: CSE-1230

Course Title: Competitive Programming I

Topic: Array

Syllabus Contents

Arrays [7 marks]

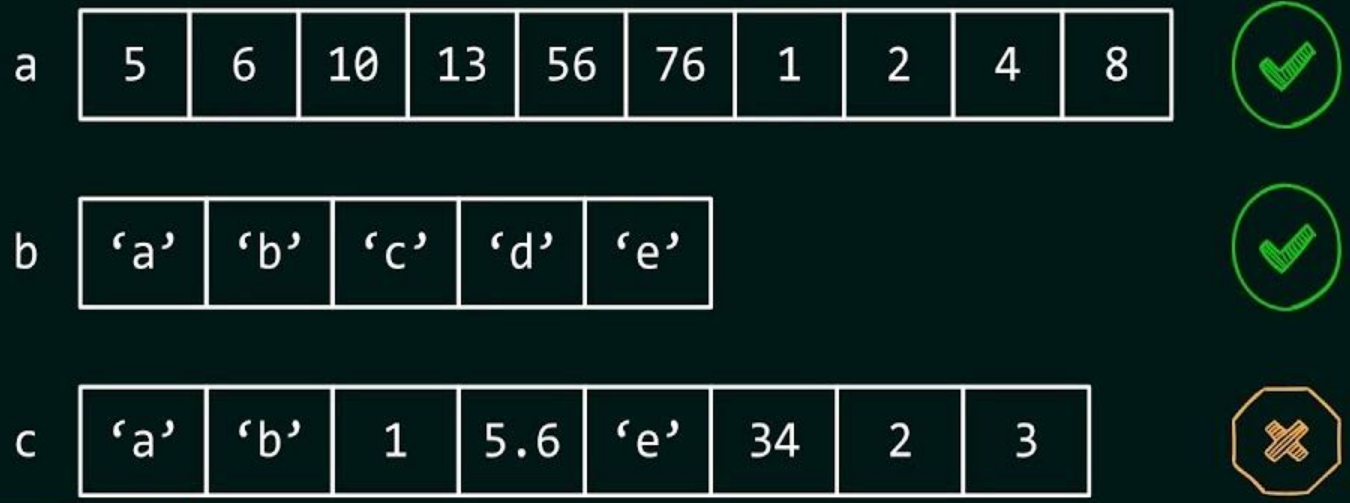
- Defining and Initializing Arrays.
- Operations: Find, Insert, Erase, Copy, Reverse, etc.
- Subaray, Subsequence, Prefix Sum, Frequency Array
- Multidimensional Arrays
- 2D Grid

Lecture – 4

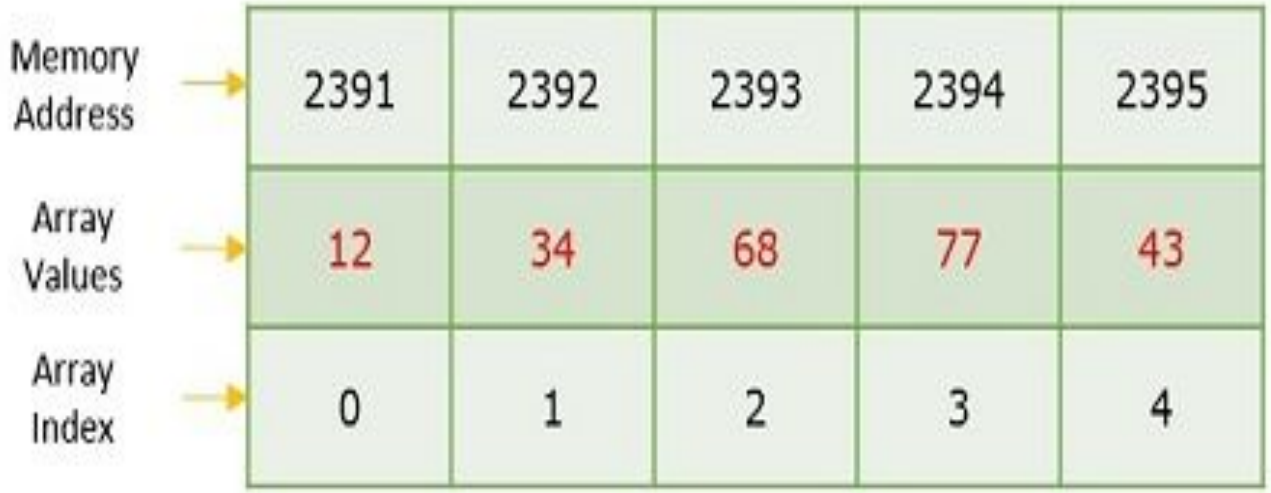
Array

Definition of Array

An Array is a data structure containing a number of data values (all of which are of same type)



Array Definition



Array Declaration
&
Initialization

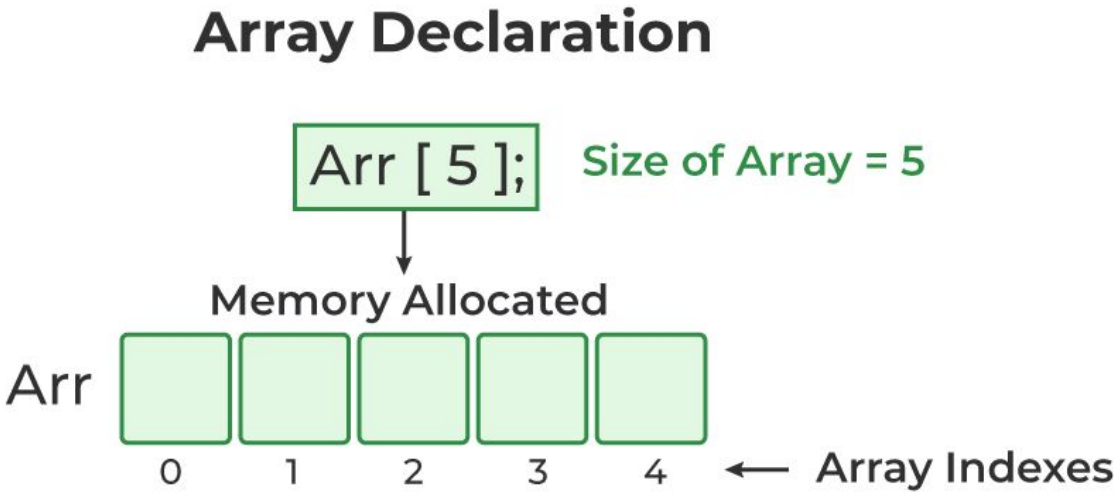
Array

- **Array** contains a number of data items of the **same type**. This type can be a simple data type such as int or float, or it can be user-defined type such as structure or a class.
- The items in an array are called **elements**. Elements are accessed by number; this number is called an **index**.

Defining an array

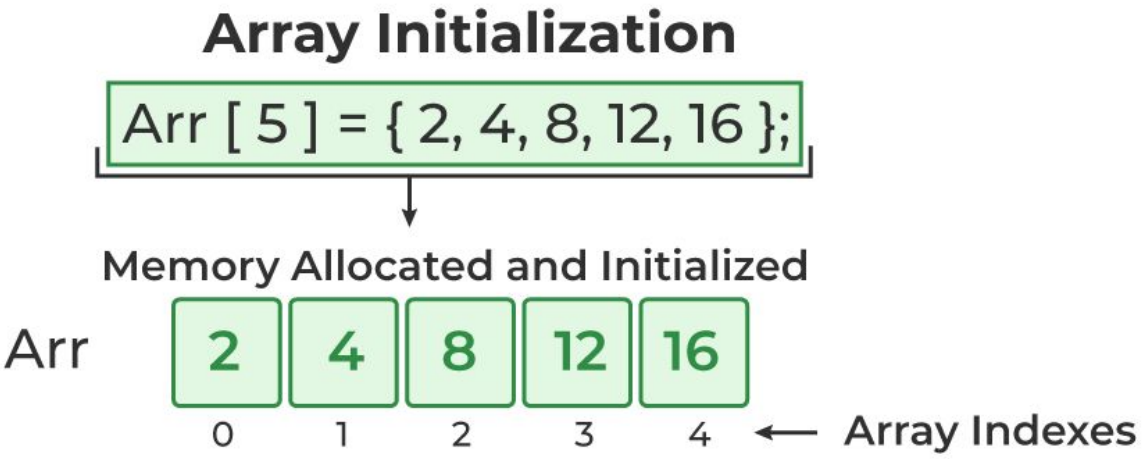
Data-type array-name [expression];

int x [100];



Array

- **Processing/ Initialization of an array**




```
//Input
scanf("%d",&n);
for(i=1 ; i<=n ; i++)
        scanf("%d",&x[i]);

//Output
for(i=1 ; i<=n ; i++)
        printf("%d ",x[i]);
```

- *Tips*: int x [110]; // make it a habit to set array size a bit larger than needed

Array vs Vector

Array Functions	Description	Vector Functions
`sizeof(arr)`, `arr[i]`, `arr[start:end]`	Accessing and sizing	`vec[i]`, `vec.size()`
`arr.push_back(x)`, `arr.pop_back()`	Adding and removing elements	`vec.push_back(x)`, `vec.pop_back()`
`arr.size()`, `arr.empty()`, `arr.clear()`	Size and emptiness checks	`vec.size()`, `vec.empty()`, `vec.clear()`
`arr[i][j]`	Accessing 2D elements	`vec[i][j]` (For vector of vectors)
`memset(arr, value, sizeof(arr))`	Sets each element in the array to a specific value.	N/A

Choosing Data Types of array

Step	Description	Example
1.	**Analyze Input Range:** Examine the range of values that the input can take based on the problem's constraints.	Values in the array are up to 10^9.
2.	**Prevent Overflow:** Select a data type that can accommodate the maximum value within the input range. This prevents overflow and ensures accurate calculations.	Since values can go up to 10^9, `int` might cause overflow.
3.	**Use int for Smaller Values:** For input sizes up to 10^5 or when values are within the `int` range, use the `int` data type.	Array size is up to 10^5, and values are within the `int` range. Using `int` is appropriate.
4.	**Use long long for Larger Values:** If the input size is larger (e.g., 10^6 or more) or values exceed the `int` range, use `long long` to avoid overflow.	Although the array size is within limits, the values can cause overflow in `int`. Using `long long` ensures accuracy.

Choosing Array Size

Step	Description	Example
1.	**Understand Problem Requirements:** Determine the maximum size of the array based on the problem statement. Be aware of any constraints.	The problem specifies that the array can have up to 10^5 elements.
2.	**Allocate Memory Optimally:** Choose an array size that meets the problem's requirements without wasting memory. Avoid overestimating the size.	Using the maximum array size (10^5) for an array with only 100 elements wastes memory.
3.	**Consider Space Complexity:** Be mindful of memory limitations. Large arrays might lead to high space complexity.	If each element occupies significant memory and the problem has strict memory limits, using a large array might not be feasible.

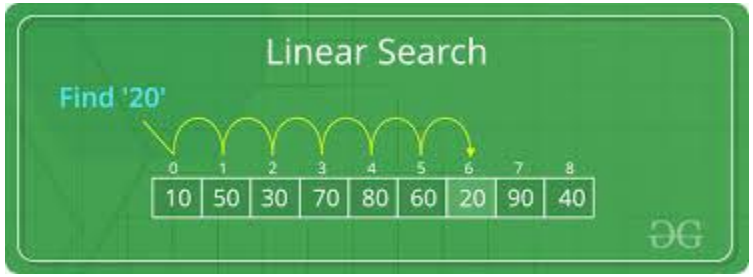
```cpp
const int MAX_SIZE = 100000; // Maximum array size
int arr[MAX_SIZE]; // Declare an array with the maximum size

int N; // Input
for (int i = 0; i < N; i++) {
    cin >> arr[i]; // Input array elements
}

int maxIndex = 0;
for (int i = 1; i < N; i++) {
    if (arr[i] > arr[maxIndex]) {
        maxIndex = i; // Update index of maximum element
    }
}

cout << "Index of maximum element: " << maxIndex << endl;
```

Operations:
Find, Insert, Erase, Copy, Reverse, etc.

Linear Search

(**Linear Search**) A linear array **DATA** with **N** elements and a specific **ITEM** of information are given. This algorithm finds the location **LOC** of **ITEM** in the array **DATA** or sets **LOC=0**.

I. Set *K := 1* and LOC := 0
2. Repeat Steps 3 arid 4 while LOC = 0 and *K* <= *N*
3. If ITEM = DATA [K], then: Set LOC := K
4. Set K := K + 1
5. If LOC = 0, then:
   Write: ITEM is not in the array DATA
   Else:
   Write: LOC is the location of ITEM
6. Exit.



Sorting: Bubble Sort

Let A be a list of *n* numbers. ***Sorting*** A refers to the operation of rearranging the elements of A so they are in increasing order, i.e., so that

A[1] < A[2] < A[3] < . . . < A[N]

For example, suppose A originally is the list

8, 4, 19, 2, 7, 13, 5, 16

After sorting, A is the list

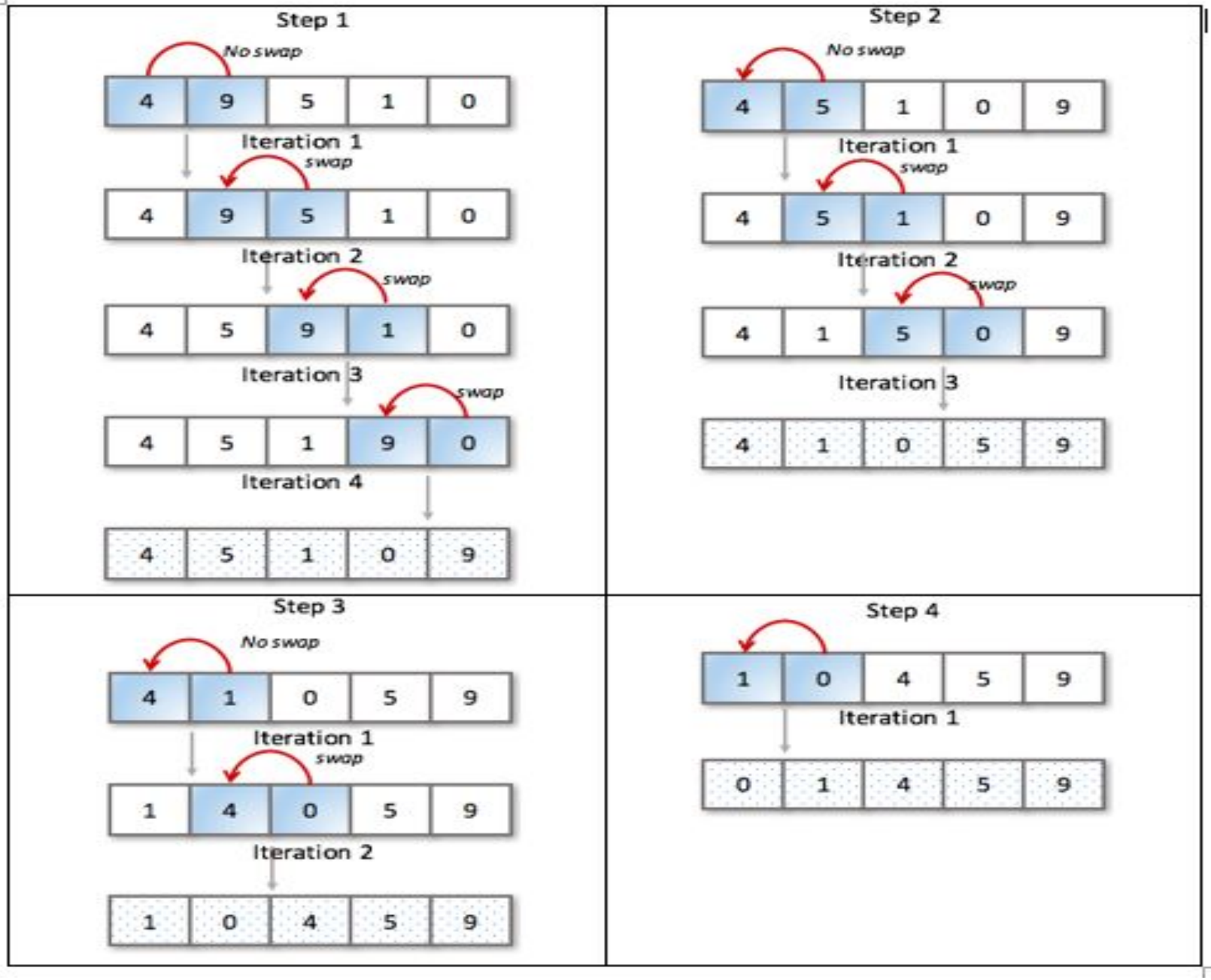
2, 4, 5, 7, 3, 13, 16, 19.

(**Bubble Sort**) BUBBLE (DATA, N)

Here **DATA** is an array with **N** elements. This algorithm sorts the elements in **DATA**.

1. Repeat Steps 2 and 3 for K = 1 to N - 1
2. Set PTR := 1
3. Repeat while PTR <= N - K:
   (a) If DATA[PTR] > DATA[PTR+1], then:
   Interchange DATA[PTR] and DATA[PTR+1]
   (b) Set PTR := PTR+1
4. Exit.

Example of Bubble Sort



Binary Search

Suppose DATA is an array which is sorted in increasing numerical order or, equivalently, alphabetically. Then there is an extremely efficient searching algorithm, called ***binary search***, which can be used to find the location LOC of a given ITEM of information in DATA.

(**Binary Search**) BINARY (DATA, LB, UB, ITEM, LOC)

Here DATA is a sorted array with lower bound LB and upper bound UB and ITEM is a given item of information. The variables BEG, END and MID denote, respectively, the beginning, end and middle locations of a segment of elements of DATA. This algorithm finds the location LOC of ITEM in DATA or sets LOC=NULL.

1. Set BEG := LB, END := UB and MID := INT((BEG+END)/2)
2. Repeat Steps 3 and 4 while BEG ≤ END and DATA [MID] ≠ ITEM
3. If ITEM < DATA [MID], then:
   Set END := MID-1
   Else:
   Set BEG := MID+1
4. Set MID := INT((BEG+END)/2)
5. If DATA [MID] = ITEM, then:
   Set LOC := MID
   Else:
   Set LOC := NULL
6. Exit.

Binary Search

Example 4.9

Let DATA be the following sorted 13-element array:

DATA: 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, 99

We apply the binary search to DATA for different values of ITEM.

(a) Suppose ITEM = 40. The search for ITEM in the array DATA is pictured in Fig. 4.6, where the values of DATA[BEG] and DATA[END] in each stage of the algorithm are indicated by circles and the value of DATA[MID] by a square. Specifically, BEG, END and MID will have the following successive values:

1. Initially, BEG = 1 and END = 13. Hence
   MID = INT[(1 + 13)/2] = 7 and so DATA[MID] = 55
2. Since 40 < 55, END has its value changed by END = MID − 1 = 6. Hence
   MID = INT[(1 + 6)/2] = 3 and so DATA[MID] = 30
3. Since 40 > 30, BEG has its value changed by BEG = MID + 1 = 4. Hence
   MID = INT[(4 + 6)/2] = 5 and so DATA[MID] = 40

We have found ITEM in location LOC = MID = 5.

(1) (11) 22, 30, 33, 40, 44, [55], 60, 66, 77, 80, 88, (99)
(2) (11) 22, [30], 33, 40, (44) 55, 60, 66, 77, 80, 88, 99
(3) 11, 22, 30, (33) [40], (44) 55, 60, 66, 77, 80, 88, 99 [Successful]

Fig. 4.6 *Binary Search for ITEM = 40*

Binary Search

(b) Suppose ITEM = 85. The binary search for ITEM is pictured in Fig. 4.7. Here BEG, END and MID will have the following successive values:

1. Again initially, BEG = 1, END = 13, MID = 7 and DATA[MID] = 55.
2. Since 85 > 55, BEG has its value changed by BEG = MID + 1 = 8. Hence

   MID = INT[(8 + 13)/2] = 10 and so DATA[MID] = 77

3. Since 85 > 77, BEG has its value changed by BEG = MID + 1 = 11. Hence

   MID = INT[(11 + 13)/2] = 12 and so DATA[MID] = 88

4. Since 85 < 88, END has its value changed by END = MID − 1 = 11. Hence

   MID = INT[(11 + 11)/2] = 11 and so DATA[MID] = 80

(Observe that now BEG = END = MID = 11.)

Since 85 > 80, BEG has its value changed by BEG = MID + 1 = 12. But now BEG > END. Hence ITEM does not belong to DATA.

(1) (11, 22, 30, 33, 40, 44, [55,] 60, 66, 77, 80, 88, (99)

(2) 11, 22, 30, 33, 40, 44, 55, (60,) 66, [77,] 80, 88, (99)

(3) 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, (80,) [88,] (99)

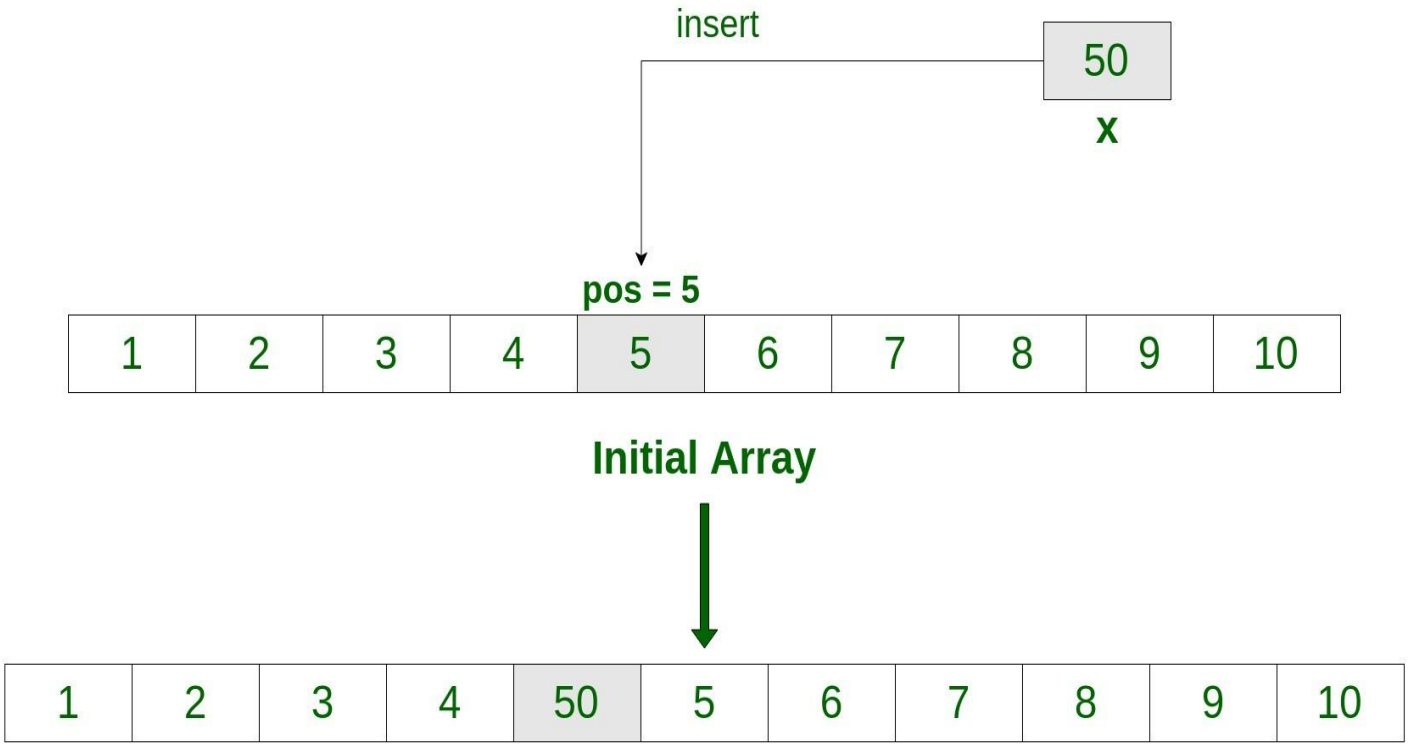
(4) 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, (80,) 88, 99 [Unsuccessful]

Fig. 4.7 *Binary Search for ITEM = 85*

Remark: Whenever ITEM does not appear in DATA, the algorithm eventually arrives at the stage that BEG = END = MID. Then the next step yields END < BEG, and control transfers to Step 5 of the algorithm.

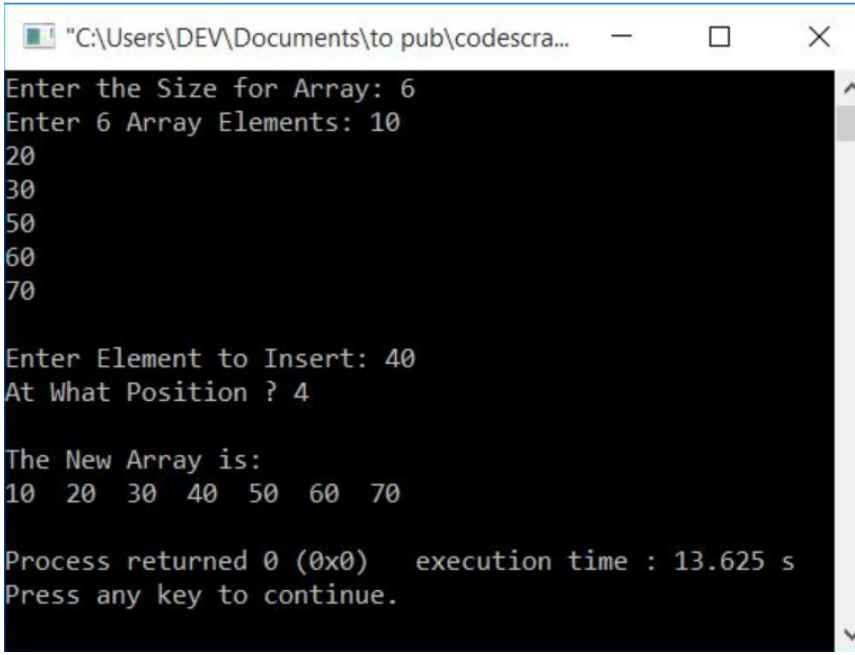
Insert an element

Insert an element at a specific position in an Array



Insert an element

```
#include<iostream>
using namespace std;
int main()
{
    int arr[50], i, elem, pos, tot;
    cout<<"Enter the Size for Array: ";
    cin>>tot;
    cout<<"Enter "<<tot<<" Array Elements: ";
    for(i=0; i<tot; i++)
        cin>>arr[i];
    cout<<"\nEnter Element to Insert: ";
    cin>>elem;
    cout<<"At What Position ? ";
    cin>>pos;
    for(i=tot; i>=pos; i--)
        arr[i] = arr[i-1];
    arr[i] = elem;
    tot++;
    cout<<"\nThe New Array is:\n";
    for(i=0; i<tot; i++)
        cout<<arr[i]<<"  ";
    cout<<endl;
    return 0;
}
```

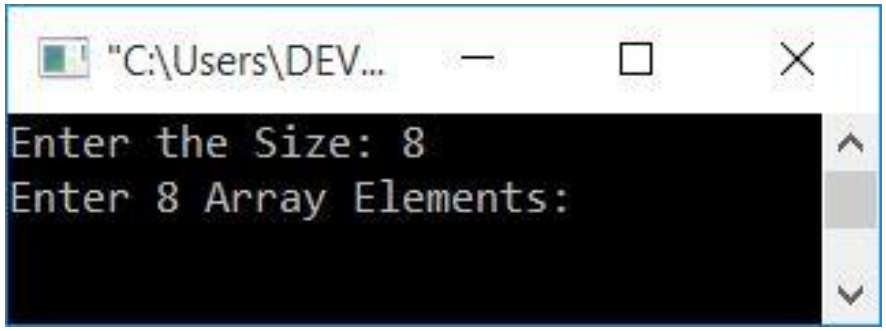


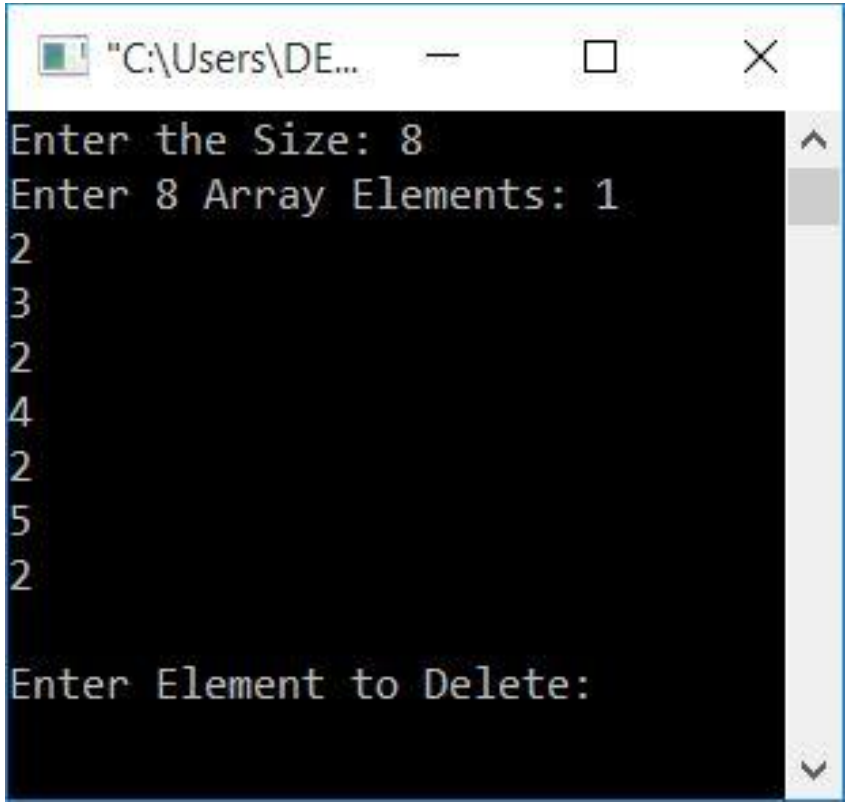
Erase

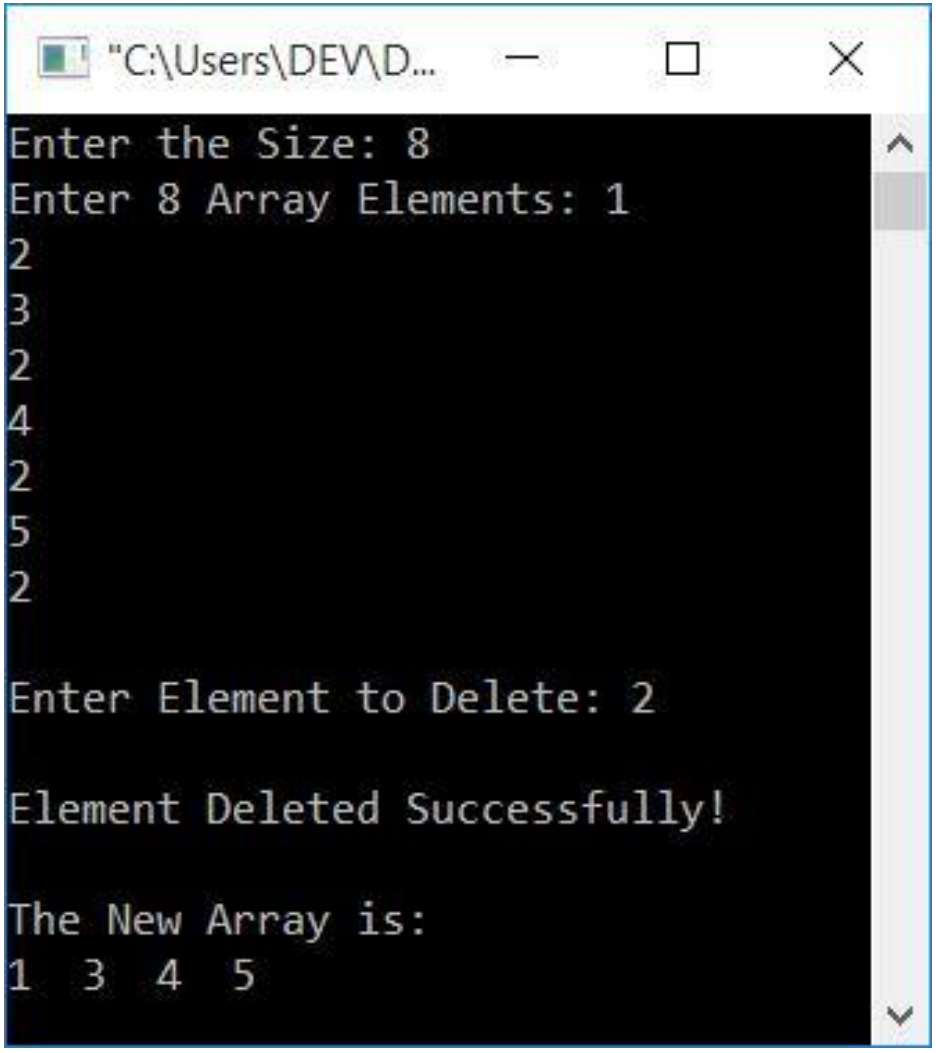
Erase an element

```
#include<iostream>
using namespace std;
int main()
{
    int arr[100], tot, i, elem, j, found=0;
    cout<<"Enter the Size: ";
    cin>>tot;
    cout<<"Enter "<<tot<<" Array Elements: ";
    for(i=0; i<tot; i++)
        cin>>arr[i];
    cout<<"\nEnter Element to Delete: ";
    cin>>elem;
    for(i=0; i<tot; i++)
    {
        if(arr[i]==elem)
        {
            for(j=i; j<(tot-1); j++)
                arr[j] = arr[j+1];
            found=1;
            i--;
            tot--;
        }
    }
    if(found==0)
        cout<<"\nElement doesn't found in the Arra
    else
    {
        cout<<"\nElement Deleted Successfully!";
        cout<<"\n\nThe New Array is:\n";
        for(i=0; i<tot; i++)
            cout<<arr[i]<<"  ";
    }
    cout<<endl;
    return 0;
}
```

Output


"C:\Users\DEV...
Enter the Size: 8
Enter 8 Array Elements:


"C:\Users\DE...
Enter the Size: 8
Enter 8 Array Elements: 1
2
3
2
4
2
5
2
Enter Element to Delete:


"C:\Users\DEV\D...
Enter the Size: 8
Enter 8 Array Elements: 1
2
3
2
4
2
5
2
Enter Element to Delete: 2
Element Deleted Successfully!
The New Array is:
1 3 4 5

Copy

Copy an element

```cpp
#include <iostream>
using namespace std;
void display( int arr[], int n ){
   for ( int i = 0; i < n; i++ ) {
      cout << arr[i] << ", ";
   }
}
void solve( int arr[], int newArr[], int n ){
   int i;
   for ( i = 0; i < n; i++ ) {
      newArr[ i ] = arr [ i ];

   }
}
 int main(){
    int arr[] = {9, 15, 24, 28, 20, 6, 12, 78, 2, 12,
    int n = sizeof( arr ) / sizeof( arr[0] );
    cout << "Given array: ";
    display(arr, n);
    int newArray[n] = {0};
    solve( arr, newArray, n );
    cout << "\nArray After copying: ";
    display(newArray, n);
```

Output

Given array: 9, 15, 24, 28, 20, 6, 12, 78, 2, 12, 78, 44, 25, 115, 255, 14, 96, 84,

Array After copying: 9, 15, 24, 28, 20, 6, 12, 78, 2, 12, 78, 44, 25, 115, 255, 14, 96, 84,

Reverse

Print array in reverse order

```
#include<stdio.h>

int main()
{
    int n, i, num[100];

    scanf("%d",&n);

    for(i=1 ; i<=n ; i++)
        scanf("%d",&num[i]);

    //printf(" Reverse Output:\n");
    for(i=n ; i>=1 ; i--)
        printf("%d\t",num[i]);

    return 0;
}
```

OUTPUT

```
3
10 20 30
30 20 10
```

Decimal to Binary

```
#include<stdio.h>
int main()
{
    int n, i=0, j, r, bin[100];

    scanf("%d",&n);

    while(n!=0)
    {
        r=n%2;
        n=n/2;
        bin[i++]=r;
    }

    for(j=i-1 ; j>=0 ; j--)
        printf("%d",bin[j]);

    return 0;
}
```

OUTPUT

13
1101

Largest Element in Array

Algorithm 2.3: (Largest Element in Array) given a nonempty array DATA with N numerical values, this algorithm finds the location LOC and the value MAX of the largest element of DATA.

1. [Initialize] Set K:=1, LOC:=1 and MAX:= DATA[1].
2. Repeat Steps 3 and 4 while K <= N:
3. If MAX < DATA [K], then:
   Set LOC: = K and MAX: = DATA [K].
4. Set K: = K+1.
   [End of Step 2 loop.]
5. Write: LOC, MAX.
6. Exit.

Functions in C++ STL

sort(arr, arr + n);

/*Here we take two parameters, the beginning of the array and the length n upto which we want the array to be sorted*/

```
//  sort in descending order
sort(arr, arr + n, greater<int>());
```

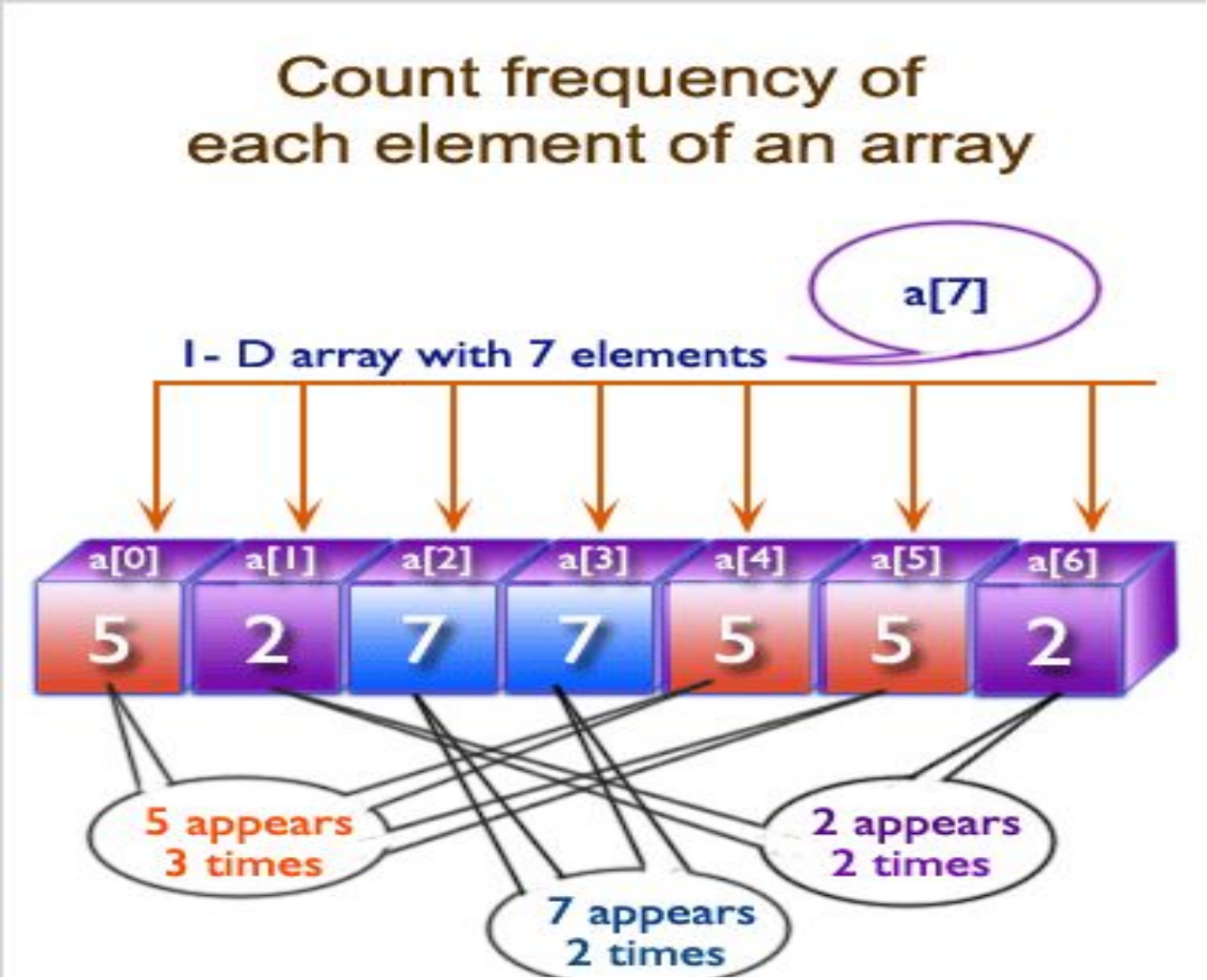
binary_search(start_ptr, end_ptr, num) : This function returns boolean **true if the element is present** in the container, else returns false.

```
// using binary_search to check if 15 exists
    if (binary_search(arr, arr+n, 15))
        cout << "15 exists in Array";
    else
        cout << "15 does not exist";

// Find the maximum element
    cout << "\nMax Element = "
         << *max_element(arr, arr + n);
```

Frequency Array

Frequency Count

Frequency count involves counting the **occurrences** of **each element** in an array.



Frequency Count with Brute Force

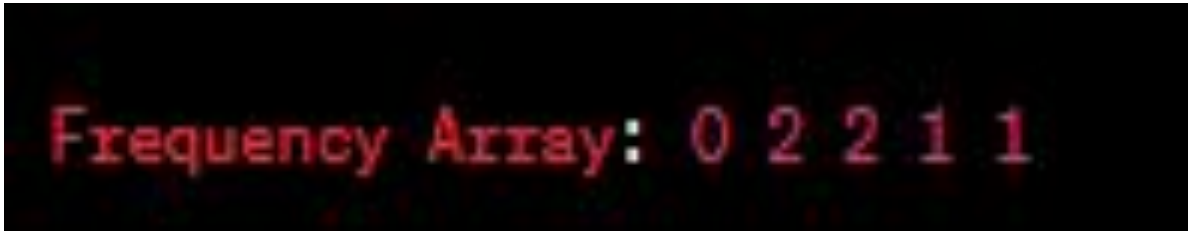
Link:

https://paste.ubuntu.com/p/HWgX7Gh5ZC/

Frequency Count with Frequency Array

Link:

https://paste.ubuntu.com/p/F4hMDd6q8s/



Code

```
// CPP program to answer queries for
frequencies
// in O(1) time.
#include <bits/stdc++.h>
using namespace std;

int hm[100];

void countFreq(int a[], int n)
{
    for(int i=0;i<n;i++)
        hm[i]=0;
    // Insert elements and their
    // frequencies in hash map.
    for (int i=0; i<n; i++)
        hm[a[i]]++;
}

// Return frequency of x (Assumes that
// countFreq() is called before)
int query(int x)
{
    return hm[x];
}
```

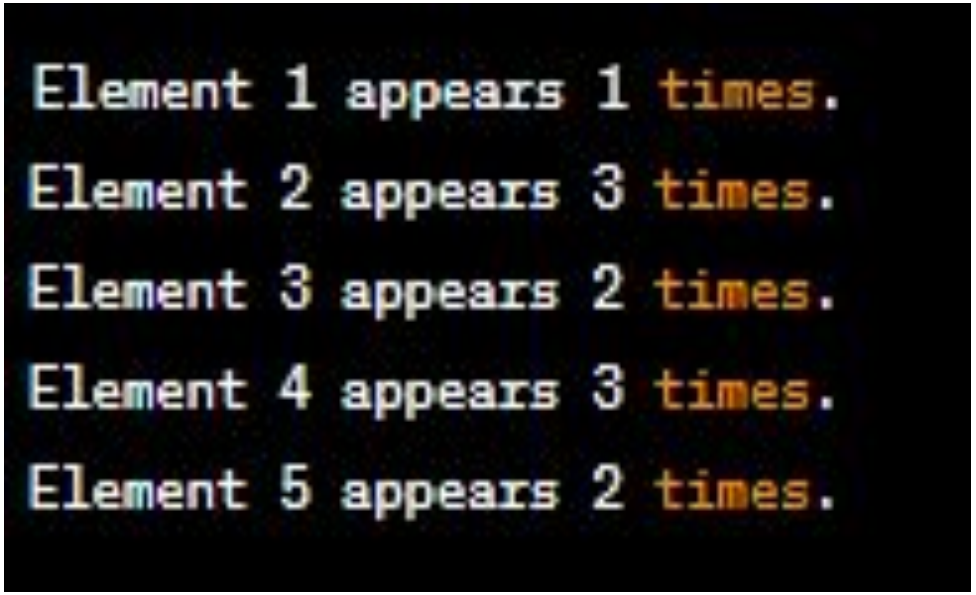
```
// Driver program
int main()
{
    int a[] = {1, 3, 2, 4, 2, 1};
    int n = sizeof(a)/sizeof(a[0]);
    countFreq(a, n);
    for (int i=0; i<n; i++){
        if(hm[i]!=0)
    cout <<"Frequency of " <<i<<" is = "
<<hm[i]<<endl;}

    return 0;
}
```

```
Frequency of 1 is = 2
Frequency of 2 is = 2
Frequency of 3 is = 1
Frequency of 4 is = 1
```

Frequency Count with STL MAP

Link:

https://paste.ubuntu.com/p/yVPFFHVzPN/



Compare Time Complexity

The brute force approach for frequency count has a time complexity of **O(N^2)** since it uses nested loops.

The frequency array approach and Map have a time complexity of **O(N)** since it iterates through the array once.

UVA Problem 11577: Letter Frequency

Problem Description:

Given a text consisting of lowercase letters, you need to count the frequency of each letter and print the letters that have the maximum frequency.

Input:

The input consists of multiple lines, each containing a non-empty string of lowercase letters. The length of the input text is at most 200 characters.

Output:

For each input text, print the letters that have the maximum frequency. If multiple letters have the same maximum frequency, print them in alphabetical order.

Logic Building:

1. Read the input text and convert all letters to lowercase for uniformity.
2. Create an array or map to store the frequency of each letter.
3. Iterate through each character in the text and update the frequency count.
4. Find the maximum frequency.
5. Print the letters that have the maximum frequency.

Pseudocode

```
function letterFrequency(text):
    Initialize a frequency array for lowercase letters
    Convert the text to lowercase

    for each character in the text:
        if character is a lowercase letter:
            increment its frequency in the array

    Find the maximum frequency

    for each lowercase letter:
        if its frequency is equal to the maximum frequency:
            print the letter
```

Homework

- https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&category=24&page=show_problem&problem=1003
- https://codeforces.com/group/MWSDmqGsZm/contest/219774/problem/V
- https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&category=24&page=show_problem&problem=1003
- https://onlinejudge.org/index.php?option=onlinejudge&Itemid=8&page=show_problem&problem=1730
- https://onlinejudge.org/index.php?option=onlinejudge&Itemid=8&page=show_problem&problem=440
- HackerRank Most Frequent

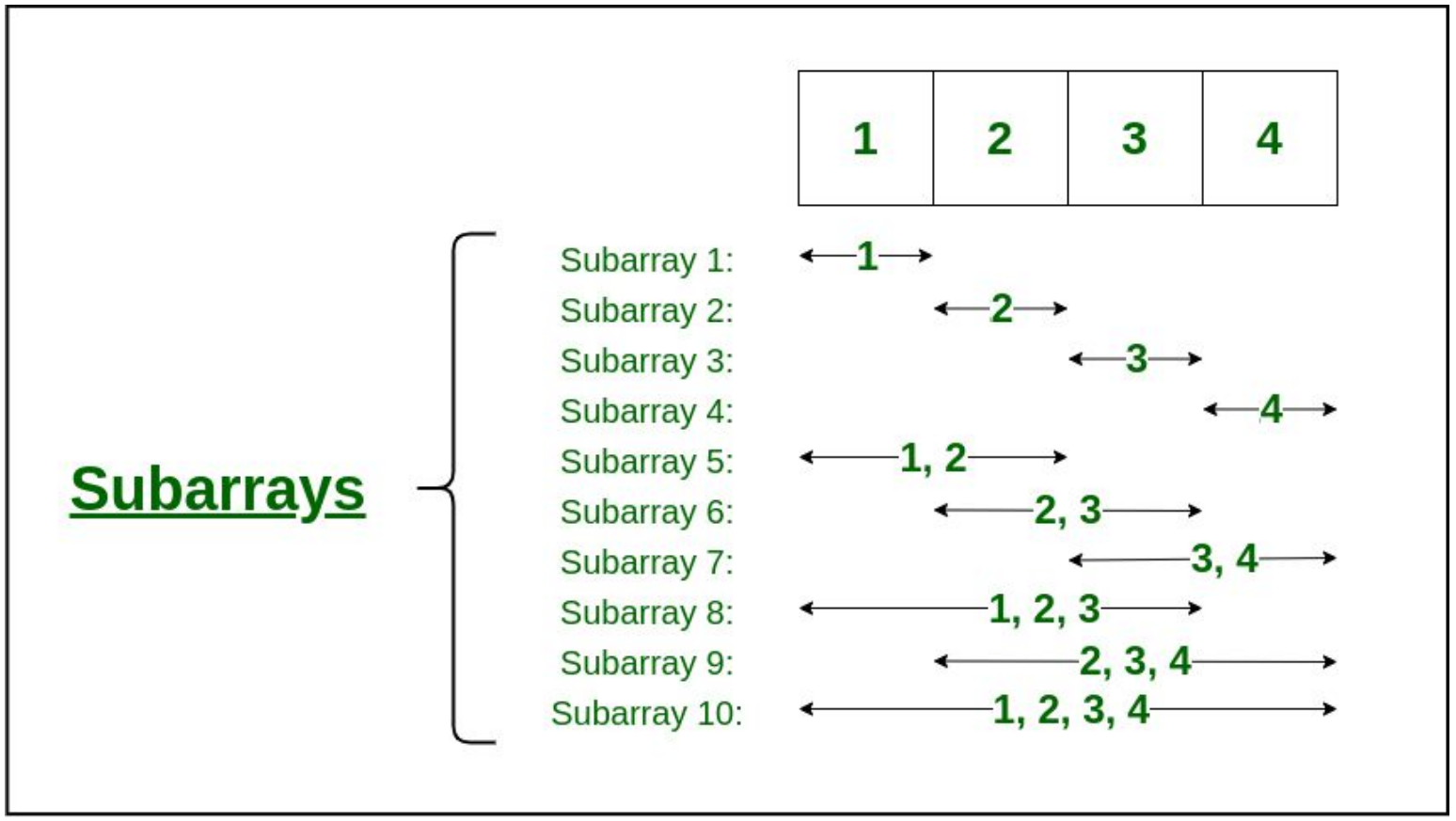
SubArray

Sub array

- A subarray is a contiguous section of an array in which the elements are consecutive and appear in the same order as they do in the original array. In other words, a subarray is formed by selecting a range of elements from the original array, where the elements are adjacent to each other.

- Example: For an array arr = [1, 2, 3, 4, 5], some possible subarrays are [1], [2, 3, 4], [3, 4, 5], and [2]. Note that [1, 3] is not a valid subarray as the elements are not adjacent.

Sub array

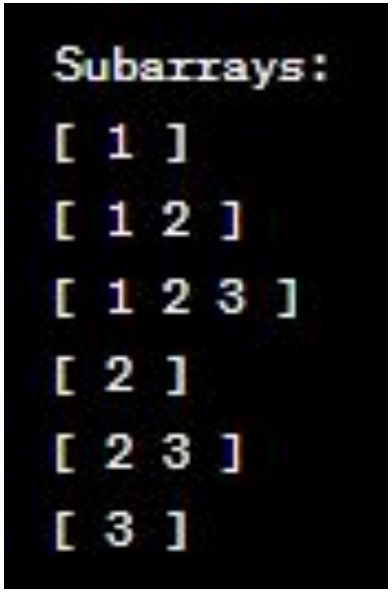

1
2
3
4
Subarrays
Subarray 1:
1
Subarray 2:
2
Subarray 3:
3
Subarray 4:
4
Subarray 5:
1, 2
Subarray 6:
2, 3
Subarray 7:
3, 4
Subarray 8:
1, 2, 3
Subarray 9:
2, 3, 4
Subarray 10:
1, 2, 3, 4

Sub array

In general, for an array/string of size n, there are n*(n+1)/2 non-empty subarrays/substrings.

How to generate all subarrays?

We can run two nested loops, the outer loop picks starting element and inner loop considers all elements on right of the picked elements as ending element of subarray.

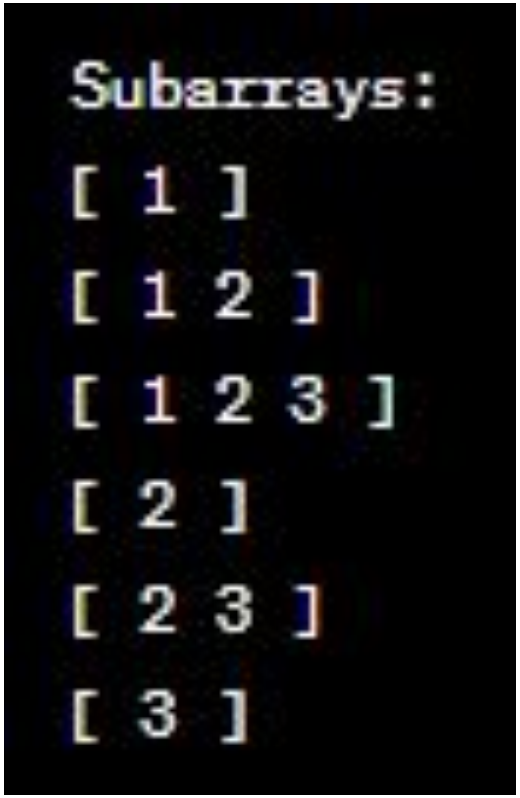


Sub array

Link: https://paste.ubuntu.com/p/vdw8pGgH5j/

Time Complexity: 0(n^3)

Space Complexity: 0(1)



SubArray Code

```cpp
/*  C++ code to generate all possible subarrays/subArrays
    Complexity- O(n^3) */
#include<bits/stdc++.h>
using namespace std;
// Prints all subarrays in arr[0..n-1]
void subArray(int arr[], int n)
{
    // Pick starting point
    for (int i=0; i <n; i++)
    {
        // Pick ending point
        for (int j=i; j<n; j++)
        {
            // Print subarray between current starting
            // and ending points
            for (int k=i; k<=j; k++)
                cout << arr[k] << " ";
            cout << endl;
        }
    }
}
```

```cpp
// Driver program
int main()
{
    int arr[] = {1, 2, 3, 4};
    int n = sizeof(arr)/sizeof(arr[0]);
    cout << "All Non-empty Subarrays\n";
    subArray(arr, n);
    return 0;
}
```

Output

Output:

```
All Non-empty Subarrays
1
1 2
1 2 3
1 2 3 4
2
2 3
2 3 4
3
3 4
4
```

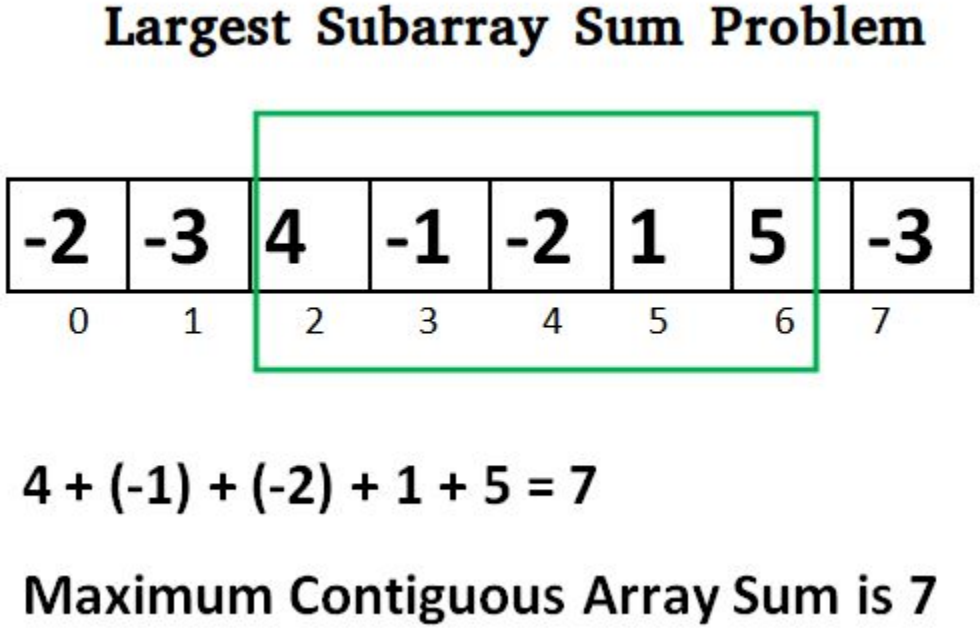
Homework

https://codeforces.com/group/MWSDmqGsZm/contest/219774/problem/L

Maximum Subarray Sum

(Kadane's Algorithm)

Given an array arr[] of size N. The task is to find the sum of the contiguous subarray within a arr[] with the largest sum.



Maximum Subarray Sum

(Kadane's Algorithm)

The idea of Kadane's algorithm is to maintain a variable ***max_ending_here*** that stores the maximum sum contiguous subarray ending at current index and a variable ***max_so_far*** stores the maximum sum of contiguous subarray found so far, Everytime there is a positive-sum value in max_ending_here compare it with max_so_far and update max_so_far if it is greater than max_so_far.

Maximum Subarray Sum

(Kadane's Algorithm)

So the main Intuition behind Kadane's algorithm is,

– *the subarray with negative sum is discarded (by assigning max_ending_here = 0 in code).*

– *we carry subarray till it gives positive sum***.**

Pseudocode

Initialize:

max_so_far = INT_MIN

max_ending_here = 0

Loop for each element of the array

(a) max_ending_here = max_ending_here + a[i]

(b) if(max_so_far < max_ending_here)

max_so_far = max_ending_here

(c) if(max_ending_here < 0)

max_ending_here = 0

return max_so_far

Illustration

Lets take the example: {-2, -3, 4, -1, -2, 1, 5, -3}

max_so_far = INT_MIN

max_ending_here = 0

for i=0, a[0] = -2

max_ending_here = max_ending_here + (-2)

Set max_ending_here = 0 because max_ending_here < 0

and set max_so_far = -2

for i=1, a[1] = -3

max_ending_here = max_ending_here + (-3)

Since max_ending_here = -3 and max_so_far = -2, max_so_far will remain -2

Set max_ending_here = 0 because max_ending_here < 0

for i=2, a[2] = 4

max_ending_here = max_ending_here + (4)

max_ending_here = 4

max_so_far is updated to 4 because max_ending_here greater than max_so_far which was -2 till now

Link

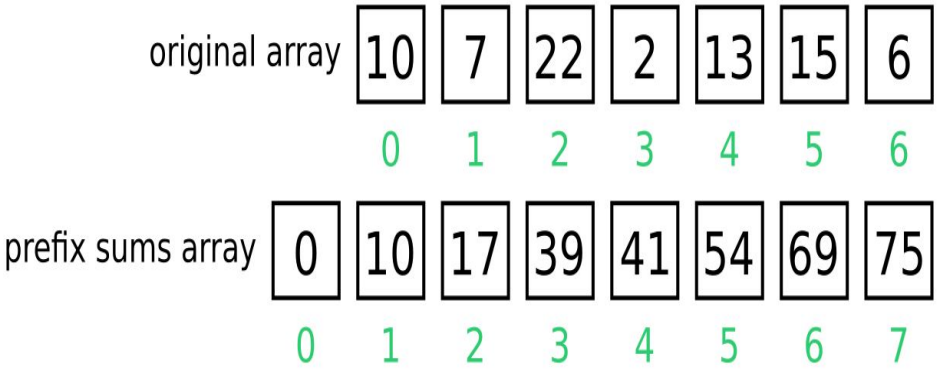
- https://www.geeksforgeeks.org/largest-sum-contiguous-subarray/

Prefix Sum

Prefix Sum

Prefix sum is a technique used to preprocess an array in such a way that we can efficiently find the sum of elements within any subarray in constant time.

Time Complexity: O(N)



Input: *arr[] = {10, 20, 10, 5, 15}*

Output: *prefixSum[] = {10, 30, 40, 45, 60}*

Explanation: *While traversing the array, update the element by adding it with its previous element.*

prefixSum[0] = 10,

prefixSum[1] = prefixSum[0] + arr[1] = 30,

prefixSum[2] = prefixSum[1] + arr[2] = 40 and so on.

Steps

Approach: To solve the problem follow the given steps:

- Declare a new array **prefixSum[]** of the same size as the input array
- Run a for loop to traverse the input array
- For each index add the value of the current element and the previous value of the prefix sum array

Prefix Sum

Link

https://paste.ubuntu.com/p/Y2wYzCYk25/

homework

- https://codeforces.com/group/MWSDmqGsZm/contest/219774/problem/Y
- https://www.spoj.com/problems/CSUMQ/
- https://codeforces.com/contest/433/problem/B
- https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&category=24&page=show_problem&problem=44

SubSequence

Subsequence

- A subsequence is a sequence of elements from an array in which the elements are not necessarily adjacent but appear in the same order as they do in the original array. In other words, a subsequence is formed by selecting some elements from the original array, where the order of the elements is preserved, but there may be gaps between them.

- Example: For an array arr = [1, 2, 3, 4, 5], some possible subsequences are [1], [1, 3, 5], [2, 4, 5], and [1, 2, 4]. Note that [1, 4] is also a valid subsequence as the order of elements is preserved, but there are gaps between them.

Approach

How to generate all Subsequences?

We can use **algorithm to generate power set** for generation of all subsequences.

Algorithm to Generate Power Set

Algorithm:

Input: Set[], set_size

1. Get the size of power set

   powet_set_size = pow(2, set_size)

2 Loop for counter from 0 to pow_set_size

   (a) Loop for i = 0 to set_size

      (i) If ith bit in counter is set

         Print ith element from set for this subset

   (b) Print separator for subsets i.e., newline

Subsequence Code

```
/*  C++ code to generate all possible subsequences.
  Time Complexity O(n * 2^n) */
#include<bits/stdc++.h>
using namespace std;

void printSubsequences(int arr[], int n)
{
  /* Number of subsequences is (2**n -1)*/
  unsigned int opsize = pow(2, n);

 /* Run from counter 000..1 to 111..1*/
  for (int counter = 1; counter < opsize; counter++)
  {
    for (int j = 0; j < n; j++)
    {
      /* Check if jth bit in the counter is set
        If set then print jth element from arr[] */
      if (counter & (1<<j))
        cout << arr[j] << " ";
    }
    cout << endl;
  }
}
```

```
int main()
{
  int arr[] = {1, 2, 3, 4};
  int n =
sizeof(arr)/sizeof(arr[0]);
  cout << "All Non-empty
Subsequences\n";
  printSubsequences(arr, n);
  return 0;
}
```

SubSequence

Consider the array `arr = [1, 2, 3, 4, 5]`.

- `[1]`
- `[1, 3, 5]`
- `[2, 4, 5]`
- `[1, 2, 4]`
- `[1, 2, 5]`
- `[2, 3, 4, 5]`

Subarray Vs Subsequence

A subarray or substring will always be contiguous, but a subsequence need not be contiguous. That is, subsequences are not required to occupy consecutive positions within the original sequences. But we can say that both contiguous subsequence and subarray are the same.

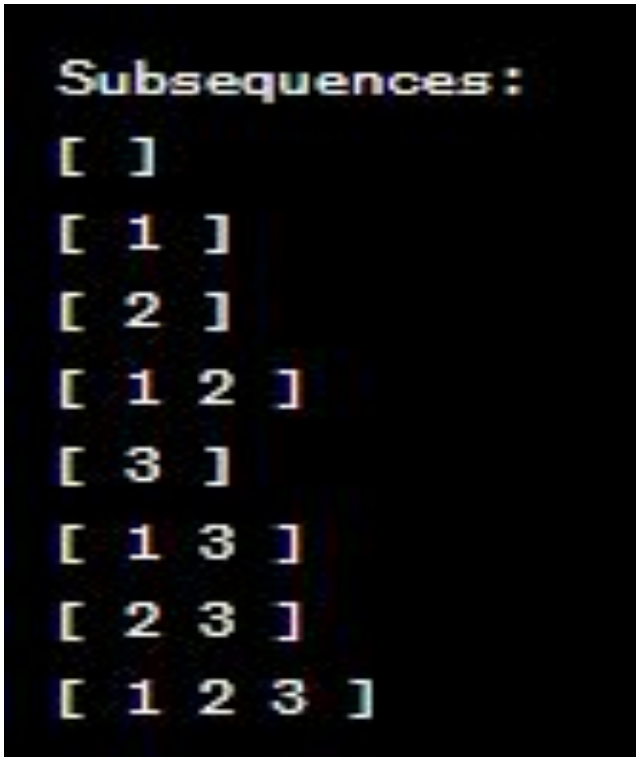
SubSequence

Link

https://paste.ubuntu.com/p/g9m5J8JNHV/

Time Complexity: 0(n*(2^n))

Space Complexity: 0(1)



mask=8

0 1 2 3 4 5 6 7

arr={1,2,3}

0-000-[]

1-001 -- [1]

2-010– [2]

3-011-[1,2]

4-100-[3]

5-101—[1,3]

6-110—[2,3]

7-111---[1,2,3]

Homework

https://codeforces.com/group/MWSDmqGsZm/contest/219774/problem/U

Use Cases

Subarray: Subarrays are commonly used to perform range-based operations, such as finding the sum, maximum, or minimum of a specific range of elements in an array.

Subsequence: Subsequences are useful for problems where you need to find patterns or combinations of elements in an array while maintaining their relative order.

Multi-dimensional Arrays

Multidimensional Array

1-D array

```
arr = [1, 2, 3, 4, 5]
```

2-D array

```
arr = [
        [11, 12, 13]
        [21, 22, 23]
        [31, 32, 33]
        ]
```

3-D array

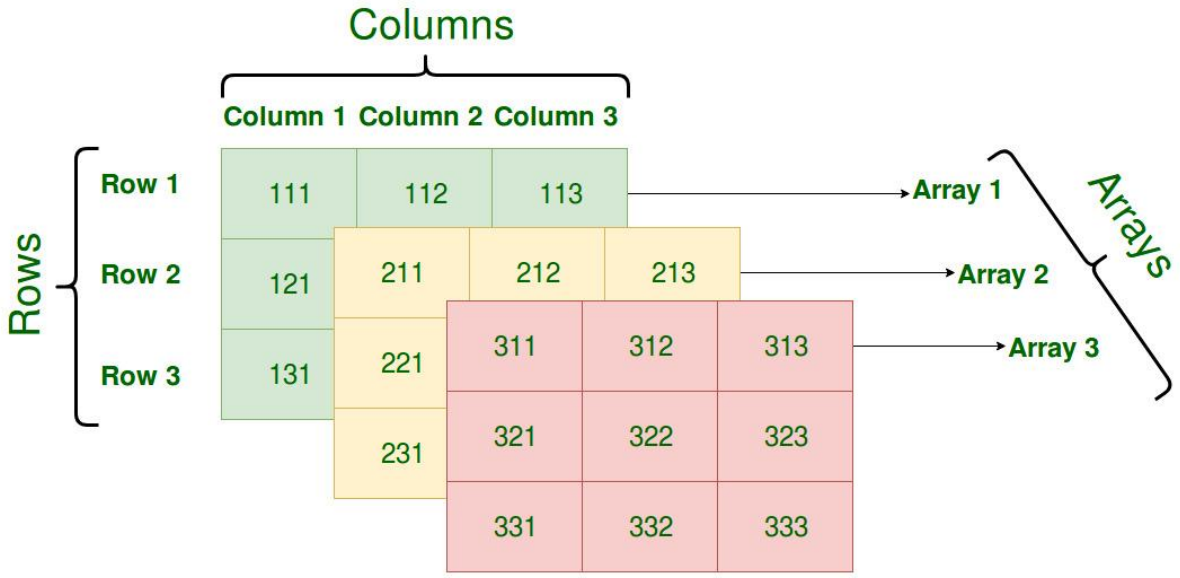
```
arr = [
        [[111, 112, 113], [121, 122, 123], [131, 132, 133]]
        [211, 212, 213], [221, 222, 223], [231, 232, 233]
        [311, 312, 313], [321, 322, 323], [331, 332, 333]]
        ]
```

	Col 1	Col 2	Col 3	Col 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]

	Col 1	Col 2	Col 3
Row 1	2	4	5
Row 2	9	0	19

Multi-dimensional Arrays

- A multi-dimensional array can be termed as an array of arrays that stores homogeneous data in tabular form.
- Data in multidimensional arrays is generally stored in row-major order in the memory.



Multi-dimensional Arrays

The ***general form of declaring N-dimensional arrays*** is shown below.

```
data_type array_name[size1][size2]....[sizeN];
```

- **data_type**: Type of data to be stored in the array.
- **array_name**: Name of the array.
- **size1, size2,..., sizeN**: Size of each dimension.

Examples:

```
Two dimensional array: int two_d[10][20];

Three dimensional array: int three_d[10][20][30];
```

Size of Multi-dimensional Arrays

The total number of elements that can be stored in a multidimensional array can be calculated by multiplying the size of all the dimensions.

For example:

- The array **int x[10][20]** can store total (10*20) = 200 elements.
- Similarly array **int x[5][10][20]** can store total (5*10*20) = 1000 elements.

To get the size of the array in bytes, we multiply the size of a single element with the total number of elements in the array.

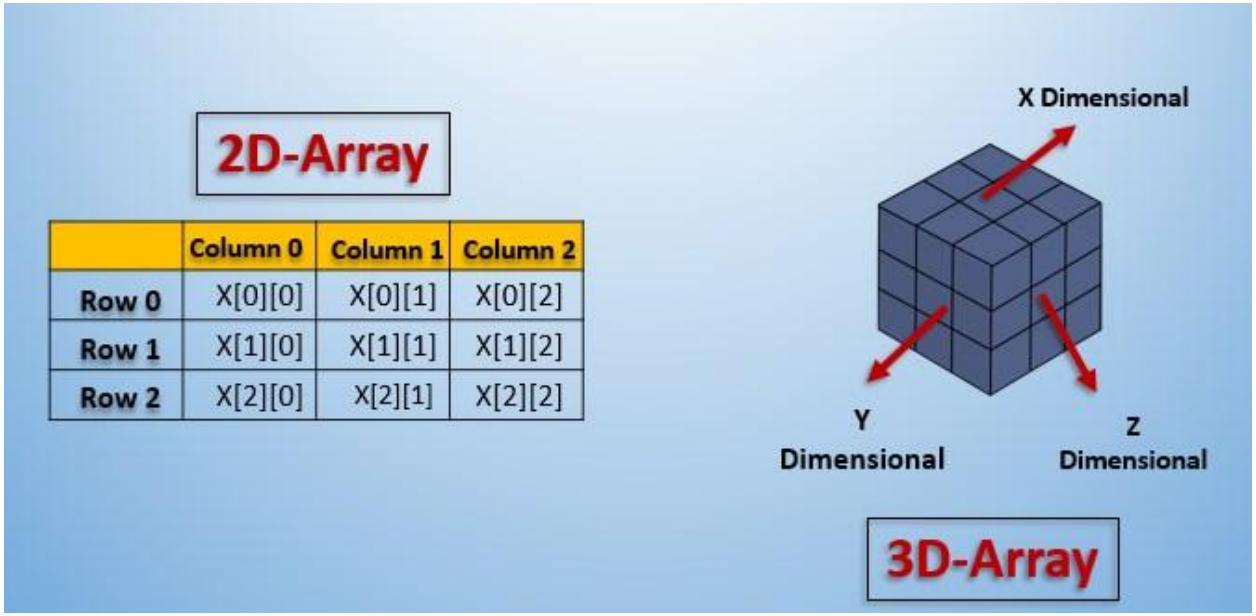
For example:

- Size of array **int x[10][20]** = 10 * 20 * 4 = 800 bytes. (where int = 4 bytes)
- Similarly, size of **int x[5][10][20]** = 5 * 10 * 20 * 4 = 4000 bytes. (where int = 4 bytes)

Forms

The most commonly used forms of the multidimensional array are:

- Two Dimensional Array
- Three Dimensional Array



2d Grid

2d grid

- A 2D grid is a two-dimensional array, typically representing a rectangular grid of elements.

Virtual Representation of 2D Grid:

- In memory, a 2D grid is stored as a one-dimensional array (vector) of vectors.

How Do We Access Them?

- Elements in a 2D grid can be accessed using row and column indices, like grid[row][col].

- **Time Complexity: O(n*m)**

2d grid

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int rows = 3;
    int cols = 4;

    // Create a 2D grid with 3 rows and 4 columns, initialized with zeroes
    vector<vector<int>> grid(rows, vector<int>(cols, 0));

    // Modify elements in the grid
    grid[0][1] = 5;
    grid[2][3] = 8;

    // Print the grid
    cout << "2D Grid:" << endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << grid[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Array of vectors

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> v[10]; // Declare an array of 10 vectors of integers

    // Accessing and modifying elements of vectors in the array
    v[0].push_back(1);
    v[0].push_back(2);
    v[1].push_back(3);
    v[2].push_back(4);

    // Displaying the contents of each vector
    for (int i = 0; i < 10; ++i) {
        cout << "Vector " << i << ": ";
        for (int num : v[i]) {
            cout << num << " ";
        }
        cout << endl;
    }

    return 0;
}
```

```
Vector 0: 1 2
Vector 1: 3
Vector 2: 4
Vector 3:
Vector 4:
Vector 5:
Vector 6:
Vector 7:
Vector 8:
Vector 9:
```

Explanation

vector<int> v[10];

This declaration creates an array of 10 vectors, where each vector can store integers. In other words, v is an array of size 10, and each element of the array is a vector that can hold integers. You can think of this as a 1D array of vectors.

2D vector

Link:

https://pastebin.ubuntu.com/p/FXf8pBpYVt/

```
2D Vector Contents:
1 2 3
4 5
Element at row 0, column 1: 2
Element at row 1, column 0: 4
```

Explanation

The output displays the individual elements accessed from the 2D vector and then prints the contents of the entire 2D vector. The 2D vector contains two rows. The first row has elements 1, 2, and 3, and the second row has elements 4 and 5. Each row is displayed on a separate line, with the elements separated by spaces.

2D array of vectors

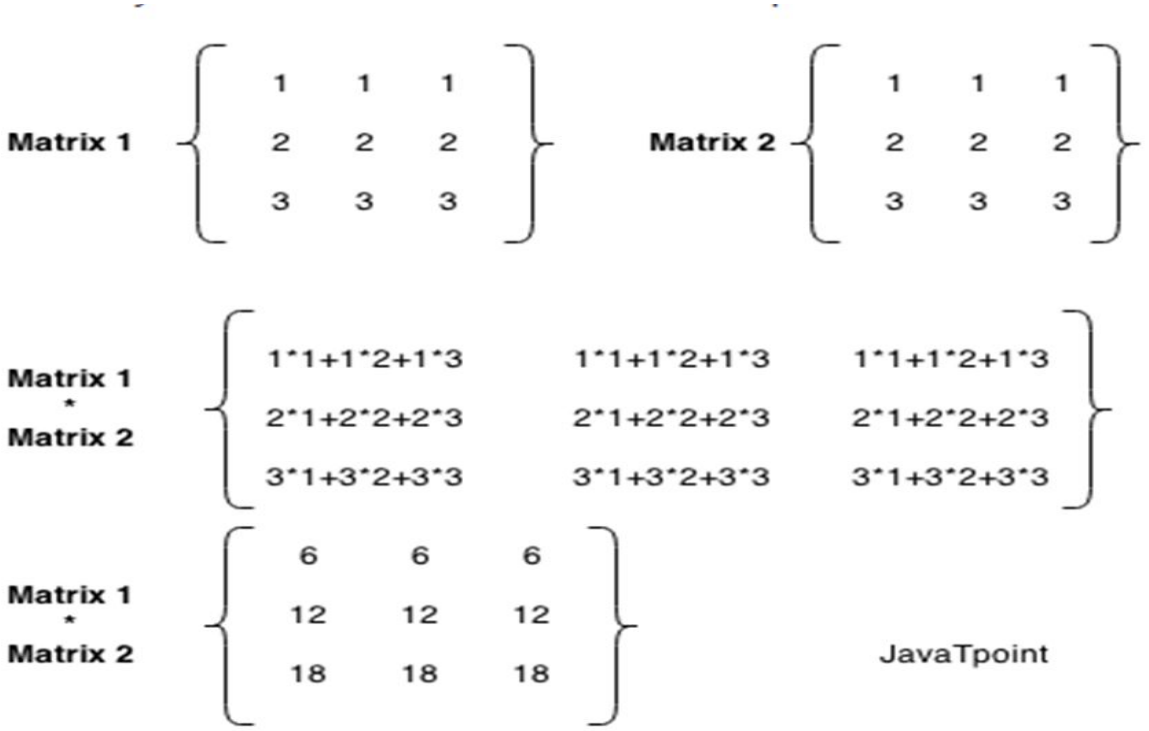
```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<vector<int>> v[3][3]; // Declare a 3×3 array of vectors of int

    // Accessing and modifying elements of vectors in the array
    v[0][0].push_back(1);
    v[1][1].push_back(2);
    v[1][1].push_back(3);
    v[2][0].push_back(4);

    // Displaying the contents of each vector
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            cout << "Vector at (" << i << ", " << j << "): ";
            for (int num : v[i][j]) {
                cout << num << " ";
            }
            cout << endl;
        }
    }

    return 0;
}
```

```
Vector at (0, 0): 1
Vector at (0, 1):
Vector at (0, 2):
Vector at (1, 0):
Vector at (1, 1): 2 3
Vector at (1, 2):
Vector at (2, 0): 4
Vector at (2, 1):
Vector at (2, 2):
```

Explanation

This declaration creates a 2D array of vectors, where each element of the array is a vector that can store integers. The array itself has dimensions 3x3, meaning it contains 3 rows and 3 columns of vectors. Each vector can store integers.

Matrix multiplication




```
for(i=0;i<r;i++)
{
   for(j=0;j<c;j++)
 {
      mul[i][j]=0;
      for(k=0;k<c;k++)
      {

mul[i][j]+=a[i][k]*b[k][j];
      }
  }
}
```

Homework

https://codeforces.com/group/MWSDmqGsZm/contest/219774/problem/X

Home work

Codeforces Sheet-3

[https://codeforces.com/group/MWSDmqGsZm/contest/219774]

UVA: 591, 10038, 10050

482, 10591, 11678, 10530,

467, 665, 11093, 11222,

11496, 11608, 11850, 12150

541, 11716, 490

11349, 11360, 11835, 10855,

10443, 11385, 12364

10374, 10508

Problems

https://onlinejudge.org

UVA 591
[Box of Bricks]

UVA 591 - Box of Bricks

```
while (1)
 {
      scanf("%d",&n);
      if(n==0) break;
      for(i=0; i<n; i++)
         scanf("%d",&a[i]);

      sum=0;
      for(i=0; i<n; i++)
         sum+=a[i];
      avg=sum/n;

      move=0;
      for(i=0; i<n; i++)
         if (…)
         {
            x = a[i]-avg;
            move+=x;
         }
        --
}
```

https://onlinejudge.org

UVA 10038
[Jolly Jumpers]

UVA 10038 - Jolly Jumpers

```
while((scanf("%d",&n)==1))
{
      for(i=0; i<=3000; i++)
      {
         a[i]=0;
         dif[i]=0;
      }

       for(i=1; i<=n; i++)
         scanf("%d",&a[i]);

       if (n==1)
           printf("Jolly\n");
       else
       {
           flag=1;
           for (i=1; i<n; i++)
           {
               d = labs(a[i]-a[i+1]);

               if((…)
                   dif[d]=1;
           }
```

UVA 10038 - Jolly Jumpers

```
for (i=1; i<n; i++)
  {
        if (dif[i]!=1)
        {
           flag=0;
           printf("Not jolly\n");
           break;
        }
      }

      if (flag==1)
         printf("Jolly\n");
   }
}
```

https://onlinejudge.org

UVA 10050

[Hartals]

UVA 10050 - Hartals

```
while(scanf("%d",&n)==1)
   {
      for (i=0; i<n; i++)
      {
         scanf("%d%d",&d,&p);
         for(a=1; a<=3650; a++)
            c[a]=0;
         for(j=0; j<p; j++)
         {
            scanf("%d",&h);
            k=h;
            while(d>=k)
            {
               c[k]=1;
               k=k+h;
            }
         }
```

UVA 10050 - Hartals

```
        ans=0;
        for(l=1; l<=d; l++)
        {
            if ((l%7==6)||(l%7==0))
                continue;
            if (c[l]==1)
                ans++;
        }

        printf("%d\n",ans);
    }
}
```

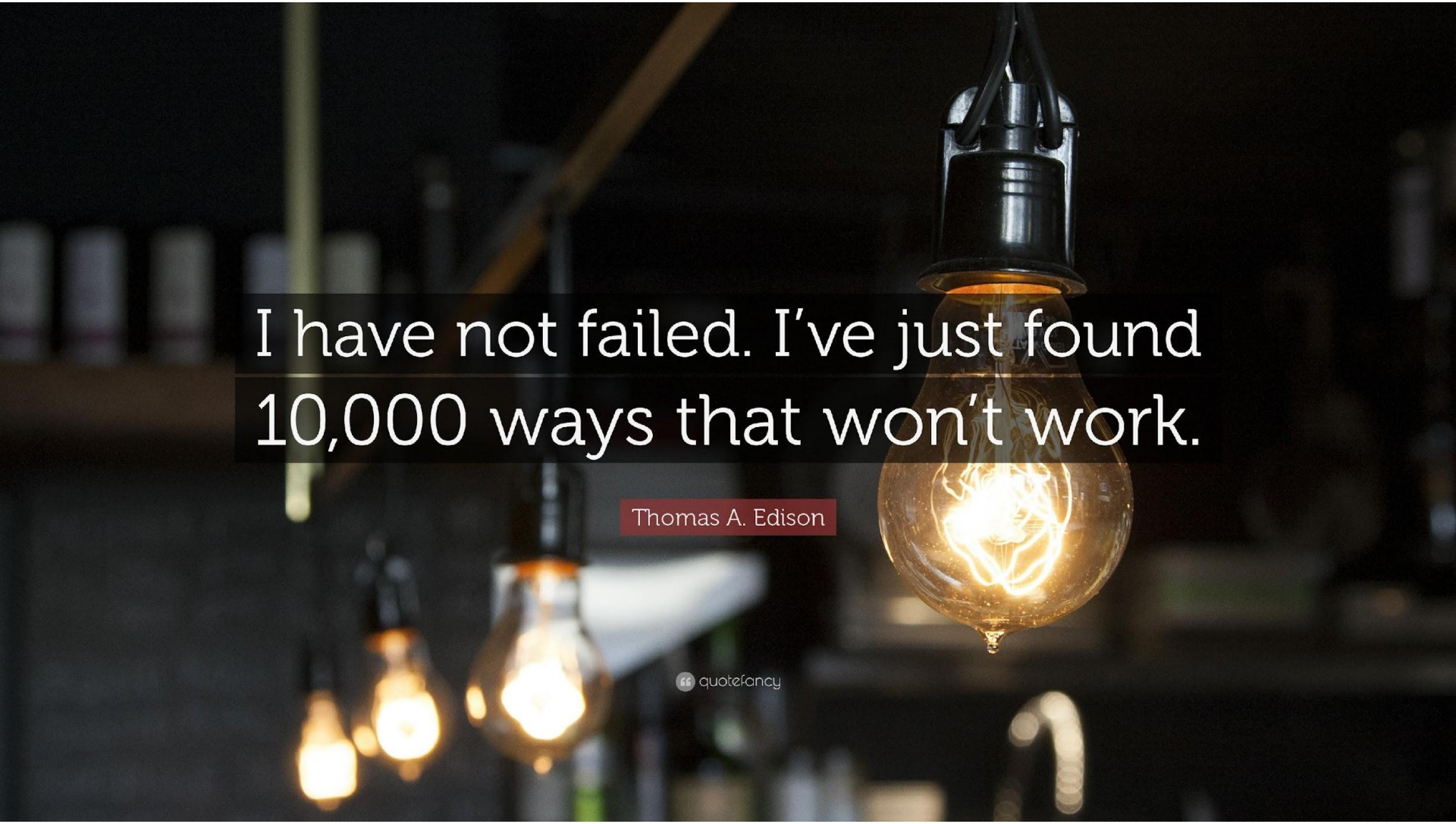
Home work

Codeforces Sheet-3 [solve A to P (except L)]
[https://codeforces.com/group/MWSDmqGsZm/contest/219774]

&

solve at least 5 problems from below list:

UVA: 591, 10038, 10050
482, 10591, 11678, 10530,
467, 665, 11093, 11222,
11496, 11608, 11850, 12150


I have not failed. I've just found
10,000 ways that won't work.
Thomas A. Edison
quotefancy

```
{
cout<<
“Happy Coding !!!”
<<endl;
}
```